

Phase D - Secret Lifecycle Analysis

Task CSE-AWS-L-013-04 | Cloud Credential Types, Secret Lifecycle Management, and Exposure Risks

Scope note: The supplied TruffleHog evidence confirmed no verified or unverified live secrets. Therefore, this Phase D evidence analyses the synthetic credential patterns used in the authorised lab as representative exposure scenarios; it does not claim discovery of a real credential.

1. Secret Lifecycle Diagram

1. CREATE	Developer key / automation token / workload role
2. STORE	Repository, .env/config, CI variable, or managed secret store
3. USE	Application, CLI, CI/CD pipeline, or AWS API authentication
4. DETECT / MONITOR	TruffleHog, GitLeaks, code review, CloudTrail
5. ROTATE	Issue replacement; update consumers; verify old secret no longer needed
6. REVOKE	Disable/delete old credential; invalidate sessions where applicable

Critical exposure points: creation without ownership, plaintext storage in source control, over-broad use, missing rotation, and delayed revocation after disclosure.

2. Lifecycle Assessment Matrix

Secret / Scenario	Creation Source	Storage	Rotation	Primary Weak Point
IAM access key pattern	Developer / IAM user	Git / .env	Manual or none	Long-lived; history persists
Automation/API token	CI/CD or service integration	Source/config/CI variable	Often manual	Reuse and broad scope
Database password	Application / DBA	Config/.env	Often irregular	Plaintext exposure
STS / role session	AWS STS / instance role	Memory/runtime metadata	Automatic expiry	Theft during validity window

3. Risk Matrix

Asset / Exposure	MITRE	L	I	Score	Rating
IAM access key committed to Git	T1552.001 / T1078	4	5	20	Critical
Hardcoded automation/API token	T1552.001	4	4	16	Critical
Plaintext DB/config secret	T1552.001	3	4	12	High
Captured temporary STS credential	T1078	2	4	8	Medium

Scoring: Likelihood x Impact. 1-4 Low | 5-9 Medium | 10-14 High | 15-25 Critical.

4. Recommendations

- **Immediate revocation:** Treat any confirmed committed credential as compromised. Disable or rotate it before repository cleanup.
- **Managed storage:** Store application secrets in AWS Secrets Manager or SSM Parameter Store and protect them with KMS rather than embedding values in source code.
- **Short-lived access:** Prefer IAM roles and STS temporary credentials for workloads and automation where feasible.
- **Least privilege:** Scope each secret/identity to the minimum required actions and resources; avoid shared credentials.
- **Automated rotation:** Define ownership and rotation intervals, and automate rotation for supported secret types.
- **Layered detection:** Run GitLeaks and TruffleHog in pre-commit/CI pipelines and manually validate high-confidence findings.
- **History cleanup:** After revocation, remove exposed material from Git history and downstream artefacts; deletion does not substitute for credential rotation.
- **Monitoring:** Use CloudTrail and alerting to investigate use of a credential after suspected exposure.

5. Residual Risk

Even with managed storage and automated scanning, secrets can still be exposed through logs, developer endpoints, CI output, or runtime compromise. Temporary credentials reduce persistence but remain usable during their active window. Controls should therefore combine prevention, detection, rapid revocation, and least privilege.